

Sistemas Embebidos

Material de la materia

UADE

UNIVERSIDAD ARGENTINA

**TP integrador de la materia: Blue Pill
como interfaz física de robot —
protocolo serie, periféricos y reacción
autónoma**

Dr. Ing. Leonardo J. Amet

Referencia de bloque: trabajo práctico integrador final. La Blue Pill actúa como interfaz física de un robot cuadrúpedo Unitree (Go2 o G1), comunicándose a través de un bridge ROS 2 provisto por la cátedra mediante un protocolo serie estructurado sobre UART.

1 Objetivos del trabajo práctico

Al terminar este TP, el grupo debería poder:

- Implementar un protocolo serie con tramas, checksum XOR y parser incremental sobre UART en Blue Pill.
- Integrar periféricos — ADC, PWM, EXTI, LED y buzzer — en un único proyecto firmware que responda a estímulos físicos reales.
- Elegir una variante cerrada de interfaz hombre-máquina o monitoreo autónomo e implementarla completa sobre el núcleo común de periféricos.
- Producir evidencia concreta de funcionamiento: tramas en monitor serie, comportamiento visible de periféricos y reacción del robot a través del bridge de la cátedra.
- Explicar las decisiones de diseño del firmware y diagnosticar fallas típicas de comunicación serie en un sistema físico real.

2 Introducción

Este es el TP final de la materia. No se trata de un ejercicio aislado: vas a tomar todo lo que aprendiste sobre build bare-metal, inicialización de periféricos, interrupciones y timers, y lo vas a poner a funcionar en un sistema que interactúa con un robot real.

La Blue Pill no controla al robot directamente. Se comunica por UART con una computadora que corre un bridge ROS 2 provisto por la cátedra. Ese bridge traduce las tramas que manda la Blue Pill en comandos, tópicos y servicios de ROS 2 que el robot entiende. Vos te concentrás en el firmware de la Blue Pill: leer sensores, reaccionar a botones, generar tramas válidas y responder a los mensajes que llegan del bridge.

El TP se resuelve en **grupos de hasta tres integrantes**. Cada grupo elige una de cinco variantes cerradas al iniciar. No hay variante libre. Todas las variantes comparten un núcleo común de periféricos y el mismo protocolo serie. Lo que cambia es qué hace la Blue Pill con esa información y cómo se comporta el robot en consecuencia.

El robot físico puede ser un **Unitree Go2 o un Unitree G1**, según disponibilidad del laboratorio. El bridge de la cátedra abstrae las diferencias entre ambos modelos. El firmware de la Blue Pill no necesita saber con cuál está hablando.

3 Prerrequisitos y material de estudio

Antes de arrancar, el grupo debe tener claro:

- **Protocolo serie ROS 2 / Blue Pill:** formato de trama, checksum XOR, parser incremental, FSM y diagnóstico. Leé el apunte de Protocolo serie ROS 2 / Blue Pill.
- **Periféricos del STM32F103:** patrón de inicialización de periféricos, RCC, GPIO, TIM, ADC, EXTI y USART. Repasá el apunte de Periféricos STM32F103.
- **UART, ADC, PWM, EXTI y timers:** son los bloques que vas a necesitar sí o sí. Revisá los apuntes correspondientes de la materia.

La cátedra va a entregar la especificación de referencia del protocolo y el bridge de prueba antes de la integración.

La implementación del TP puede hacerse sobre cualquier base de firmware funcional para Blue Pill (bare-metal con libopencm3 o con FreeRTOS). La cátedra sugiere partir de un proyecto que ya tenga UART, timers y GPIO funcionando.

4 Protocolo serie

Toda la comunicación entre la Blue Pill y el bridge ROS 2 usa el siguiente formato de trama sobre UART:

@LL:TTT:PAYLOAD:CC\n

4.1 Campos

Campo	Tamaño	Significado
@	1 byte	Marca de inicio de trama
LL	2 bytes	Longitud hexadecimal del cuerpo TTT:PAYLOAD
:	1 byte	Separador tras longitud
TTT	3 bytes	Tipo de mensaje (solo mayúsculas)
:	1 byte	Separador entre tipo y payload
PAYLOAD	variable	Contenido ASCII imprimible (máx. 48 bytes, sin @)
:	1 byte	Separador antes del checksum
CC	2 bytes	Checksum XOR en hexadecimal (calculado sobre LL:TTT:PAYLOAD)
\n	1 byte	Fin de trama

4.2 Tipos de mensaje

Tipo	Dirección típica	Uso en este TP
CMD	Bridge → Pill	Comandos del robot hacia la Pill (usado en la variante Señalización industrial)
DAT	Pill → Bridge	Datos periódicos de sensores (ADC, velocidad mapeada, parámetros)
EVT	Pill → Bridge	Eventos discretos (parada de emergencia, deadman, cambio de modo, cruce de zona)
STS	Pill → Bridge	Estado interno (heartbeat, modo activo, emergencia)
ACK	Bidireccional	Confirmación de recepción
ERR	Pill → Bridge	Error reportado (ADC fuera de rango, timeout de comunicación)

4.3 Payloads definidos para este TP

La tabla siguiente resume los payloads que vas a usar, según la variante que elijas. Todos los payloads usan ASCII imprimible, miden como máximo 12 caracteres y no contienen @.

Tipo	Payloads	Semántica
CMD	mode=RUN, mode=STOP, mode=ERROR, mode=ESTOP	Estado del robot informado a la Pill

Tipo	Payloads	Semántica
DAT	adc=NNNN, spd=NN, param=NNN	ADC crudo (4 dígitos), velocidad 0–100 (2 dígitos), parámetro 0–255 (3 dígitos)
EVT	e_stop, e_stop_rel, deadman, deadman_ok, cmd_patrol, cmd_stop, cmd_deploy, cmd_return, zone_safe, zone_warn, zone_danger	Eventos de pulsador, deadman, comando o zona
STS	OK, ALIVE, EMERGENCY, mode=PATROL, mode=STOP, mode=DEPLOY	Heartbeat, deadman activo, emergencia, modo de caja de comandos
ACK	ok, estop	Confirmación genérica, confirmación específica de parada de emergencia
ERR	bounds, timeout	ADC fuera de rango, timeout de comunicación con el bridge

4.4 Ejemplos concretos de tramas

Acá vas a encontrar cuatro ejemplos con el cálculo paso a paso de LL y checksum. Usalos como referencia para verificar tu implementación.

4.4.1. Lectura de ADC — DAT:adc=2048

```

Payload:      adc=2048
Cuerpo:      DAT:adc=2048
Longitud:    12 decimal → 0C hexadecimal
LL colocado: 0C:DAT:adc=2048
Checksum XOR: 0 ^ C ^ : ^ D ^ A ^ T ^ : ^ a ^ d ^ c ^ = ^ 2 ^ 0 ^ 4 ^ 8
              = 30 ^ 43 ^ 3A ^ 44 ^ 41 ^ 54 ^ 3A ^ 61 ^ 64 ^ 63 ^ 3D ^ 32 ^ 30 ^ 34 ^ 38
              = 77 hex → CC = 77
Trama completa: @0C:DAT:adc=2048:77\n

```

4.4.2. Evento de parada de emergencia — EVT:e_stop

```

Payload:      e_stop
Cuerpo:      EVT:e_stop
Longitud:    10 decimal → 0A hexadecimal
LL colocado: 0A:EVT:e_stop
Checksum XOR: 0 ^ A ^ : ^ E ^ V ^ T ^ : ^ e ^ _ ^ s ^ t ^ o ^ p
              = 30 ^ 41 ^ 3A ^ 45 ^ 56 ^ 54 ^ 3A ^ 65 ^ 5F ^ 73 ^ 74 ^ 6F ^ 70
              = 14 hex → CC = 14
Trama completa: @0A:EVT:e_stop:14\n

```

4.4.3. Heartbeat de estado — STS:OK

```

Payload:      OK
Cuerpo:      STS:OK
Longitud:    6 decimal → 06 hexadecimal
LL colocado: 06:STS:OK
Checksum XOR: 0 ^ 6 ^ : ^ S ^ T ^ S ^ : ^ O ^ K
              = 30 ^ 36 ^ 3A ^ 53 ^ 54 ^ 53 ^ 3A ^ 4F ^ 4B
              = 56 hex → CC = 56

```

Trama completa: @06:STS:OK:56\n

4.4.4. Comando recibido del bridge — CMD:mode=ESTOP

Payload: mode=ESTOP
Cuerpo: CMD:mode=ESTOP
Longitud: 14 decimal → 0E hexadecimal
LL colocado: 0E:CMD:mode=ESTOP
Checksum XOR: $0 \oplus E \oplus : \oplus C \oplus M \oplus D \oplus : \oplus m \oplus o \oplus d \oplus e \oplus = \oplus E \oplus S \oplus T \oplus O \oplus P$
 $= 30 \oplus 45 \oplus 3A \oplus 43 \oplus 4D \oplus 44 \oplus 3A \oplus 6D \oplus 6F \oplus 64 \oplus 65 \oplus 3D \oplus 45 \oplus 53 \oplus 54 \oplus 4F \oplus 50$
 $= 5C \text{ hex} \rightarrow CC = 5C$
Trama completa: @0E:CMD:mode=ESTOP:5C\n

Estos cuatro ejemplos cubren los tipos de mensaje más frecuentes en las variantes: dato periódico (DAT), evento de seguridad (EVT), estado interno (STS) y comando desde el bridge (CMD). Usá el mismo procedimiento para cualquier otro payload: calculás la longitud del cuerpo TTT:PAYLOAD, la pasás a hex de dos dígitos, y calculás el XOR sobre LL:TTT:PAYLOAD (sin el @ ni el \n final).

5 Seguridad y restricciones físicas

Este TP involucra un robot físico que puede moverse. Las siguientes reglas son **obligatorias y no negociables**:

1. **El bridge y la interfaz con el robot son provistos por la cátedra.** No modificar, puentear ni reconectar cables entre la computadora del bridge y el robot.
2. **No se debe intentar puentear los sistemas de seguridad del robot** (parada de emergencia nativa, sensores de colisión, límites de articulación). La Blue Pill agrega una capa de monitoreo, no reemplaza la seguridad del fabricante.
3. **La Blue Pill opera a 3.3 V.** No conectar directamente señales de 5 V a ningún pin. Si un sensor externo entrega 5 V, usar divisor resistivo o conversor de nivel.
4. **Masa común (GND):** la Blue Pill y cualquier periférico alimentado externamente deben compartir GND. La conexión GND con el robot la realiza la cátedra según las instrucciones de armado del puesto de laboratorio.
5. **Alimentación de la Blue Pill:** se recomienda alimentarla por USB desde la misma computadora que corre el bridge, o desde una fuente de laboratorio regulada a 3.3 V. No alimentar la Blue Pill desde el robot sin autorización explícita de la cátedra.
6. **Conexión UART:** el cableado UART entre la Blue Pill y la computadora del bridge (USB-UART adapter) es responsabilidad de la cátedra. Los pines USART de la Blue Pill que deben usarse son **PA9 (TX)** y **PA10 (RX)** a 115200 baudios, 8N1.

6 Consigna general

El grupo debe construir un firmware para Blue Pill que:

1. Implemente el protocolo serie completo: generación de tramas, parser incremental con FSM, validación de checksum y resincronización ante errores.
2. Integre los periféricos del núcleo común: **ADC** (lectura de sensor analógico), **PWM** (control de LED y/o buzzer), **LED** (indicador visual) y **buzzer** (indicador acústico).
3. Implemente **una** de las cinco variantes cerradas descritas más abajo.
4. Produzca evidencia concreta de funcionamiento con el bridge de la cátedra y, cuando las condiciones de laboratorio lo permitan, con el robot físico.

El trabajo avanza por cuatro etapas acumulativas. Cada etapa debe estar funcionando y documentada antes de pasar a la siguiente.

7 Etapa 1: Comunicación serie

7.1 Consigna

Implementar el protocolo serie sobre UART. Esta etapa no requiere periféricos externos más allá del USB-UART adapter para verificar tramas en la terminal.

7.2 Subtareas

1. Configurar USART1 en PA9 (TX) y PA10 (RX) a 115200 baudios, 8N1.
2. Implementar la función de generación de tramas: dado un tipo (CMD, DAT, etc.) y un payload, producir la trama completa @LL:TTT:PAYLOAD:CC\n.
3. Implementar el parser incremental con máquina de estados finitos (FSM) que consuma un byte por vez.
4. Implementar la validación final: checksum, formato TTT:PAYLOAD, tipo conocido, caracteres de payload permitidos.
5. Implementar resincronización: ante error, descartar trama parcial y volver a WAIT_START. Si el byte que causó el error es @, reutilizarlo como nuevo inicio.
6. Probar con un lazo de eco modificado: la Pill recibe una trama, la parsea, y si es válida responde con ACK:ok. Si es inválida, responde con ERR:bounds.

7.3 Preguntas obligatorias

1. ¿Por qué el parser consume un byte por vez en vez de esperar una línea completa con `readline()`?
2. ¿Qué pasa si llegan dos tramas pegadas sin separación? ¿Cómo las distingue tu parser?
3. Mostrar el cálculo de checksum para DAT:adc=2048.
4. ¿Qué hace tu implementación si recibe @0A:CMD:ping:52\n (donde 0A es incorrecto porque CMD:ping mide 8)?

7.4 Evidencia mínima

- Captura de terminal serie mostrando al menos cinco tramas enviadas y sus respuestas.
- Para al menos una trama, mostrar el recorrido completo de la FSM (estado por estado).
- Explicación de cómo se calcula LL y CC en tu código.

8 Etapa 2: Núcleo de periféricos

8.1 Consigna

Poner en funcionamiento los cuatro periféricos del núcleo común: ADC, PWM, LED y buzzer. El LED y el buzzer deben ser controlables desde el firmware; el ADC debe leer un sensor físico real.

8.2 Subtareas

1. Configurar **ADC1** en el canal que corresponda según tu variante (PA0 para potenciómetro, u otro canal para sensor provisto por la cátedra). Usar TIM3 como base temporal para lecturas periódicas.
2. Configurar al menos un canal de **PWM por hardware** (TIM2 u otro timer con salida en un pin GPIO adecuado) para controlar el brillo de un LED o la frecuencia de un buzzer.
3. Conectar un **LED** (con resistor limitador de 220 Ω a GND) al pin PWM y verificar que el brillo responde al duty cycle configurado.
4. Conectar un **buzzer** pasivo (con transistor driver si es necesario) a un pin con capacidad PWM o a un GPIO simple, según lo que requiera tu variante.
5. Si tu variante usa **EXTI** (botón), configurar el pin correspondiente con pull-down externo y anti-rebote por timer.

8.3 Preguntas obligatorias

1. Mostrar el cálculo de PSC y ARR para tu timer de PWM. ¿Qué frecuencia PWM elegiste y por qué?
2. ¿Cómo mapeás el valor ADC (0–4095) al duty cycle del PWM (0–100%)?
3. Si usás el mismo timer para PWM del LED y para frecuencia del buzzer, ¿qué limitaciones encontrás?
4. ¿Qué modo GPIO necesita un pin para funcionar como salida de PWM?

8.4 Evidencia mínima

- Tabla de configuración de periféricos (registro/API, parámetros, cálculo).
- Foto o video corto mostrando LED con brillo variable y buzzer sonando.
- Captura serie de al menos 10 lecturas de ADC mostrando valores coherentes al variar el sensor.

9 Etapa 3: Variante elegida

9.1 Consigna

Implementar la lógica específica de **una** de las cinco variantes. Esta etapa es la más extensa: acá se define qué hace la Blue Pill, qué mensajes envía, cómo reacciona a los estímulos y qué espera del bridge.

Leé la descripción completa de tu variante en la sección [Variantes cerradas](#) antes de empezar.

9.2 Qué debe funcionar al terminar esta etapa

- La Blue Pill genera las tramas que corresponden a tu variante (eventos, datos, estados).
- Los periféricos responden a los estímulos físicos según lo descrito en la variante (LED cambia patrón, buzzer cambia tono).
- Las tramas se pueden observar en el monitor serie y son sintácticamente válidas según el protocolo.
- Si tu variante recibe mensajes (CMD), la Blue Pill los parsea correctamente y ajusta su comportamiento.

9.3 Preguntas obligatorias (comunes a todas las variantes)

1. Describir el lazo principal de tu firmware: ¿qué hace en cada iteración? ¿qué corre por interrupción?
2. ¿Qué pasa si el bridge no responde con ACK dentro del tiempo de espera? ¿Cómo lo manejas?
3. ¿Cómo aseguramos que los payloads numéricos (`adc=NNNN`, `spd=NN`) siempre tengan el ancho correcto aunque el valor sea chico (ej. `adc=0023`)?
4. ¿Qué decisión tomaste sobre el uso de timers? ¿Cuántos timers usaste y para qué?

9.4 Evidencia mínima

- Diagrama de flujo o de estados de la lógica de tu variante.
- Captura de monitor serie mostrando al menos 20 segundos de operación continua con todas las tramas relevantes.
- Foto o video del montaje físico completo (Blue Pill + protoboard + periféricos).

10 Etapa 4: Integración con el robot

10.1 Consigna

Conectar la Blue Pill al bridge ROS 2 de la cátedra y verificar que las tramas producen la reacción esperada en el robot.

10.2 Subtareas

1. Conectar la Blue Pill a la computadora del bridge según las indicaciones de la cátedra (USB-UART adapter).
2. Ejecutar el bridge y verificar en los tópicos de ROS 2 que las tramas de la Blue Pill se reciben correctamente.
3. Observar la reacción del robot ante los mensajes de tu variante.
4. Probar al menos un escenario de falla: desconectar UART, enviar una trama corrupta, o forzar una condición de error de tu variante (ADC fuera de rango, timeout de ACK, etc.). Documentar cómo se comporta el sistema y cómo se recupera.

10.3 Preguntas obligatorias

1. ¿Qué tópicos de ROS 2 publica el bridge a partir de las tramas que envía tu Blue Pill?
2. ¿Qué latencia observás entre que ocurre un evento físico (apretar un botón, cruzar un umbral) y la reacción del robot?
3. En el escenario de falla que probaste, ¿la Blue Pill se recupera automáticamente o requiere intervención manual? ¿Es ese el comportamiento deseado?
4. Si tuvieras que agregar un nuevo comando de robot sin modificar el protocolo, ¿cómo lo harías?

10.4 Evidencia mínima

- Video del robot reaccionando a los estímulos generados desde la Blue Pill.
- Captura de terminal mostrando las tramas y, en paralelo, los tópicos ROS 2 publicados.
- Descripción del escenario de falla probado y del comportamiento observado.

11 Variantes cerradas

Cada grupo elige **una sola variante** al iniciar el TP. Las cinco variantes son cerradas: no se aceptan propuestas libres ni híbridos.

En todas las variantes, el bridge ROS 2 y la reacción del robot son **provistos por la cátedra**. La Blue Pill no implementa lógica ROS 2: solo genera y consume tramas por UART.

11.1 Variante 1: Parada de emergencia física

Concepto: un pulsador de golpe (tipo honguito) conectado a la Blue Pill actúa como parada de emergencia cableada. Al presionarlo, la Pill notifica al robot y activa señalización local de alarma.

Periféricos:

- **EXTI (botón):** pulsador NC o NA conectado a un pin GPIO con resistencia de pull-down externa. Anti-rebote por timer. Al presionar, la Pill entra en estado de emergencia.
- **LED:** verde fijo en operación normal; rojo con destello PWM (50 % duty, 4 Hz) en emergencia. Se recomienda usar dos LED o un LED RGB.
- **Buzzer:** tono continuo mientras la emergencia está activa.
- **ADC:** lee un potenciómetro conectado a PA0 que se usa como referencia de health check. El valor ADC se reporta periódicamente.
- **PWM:** controla el patrón de destello del LED de emergencia.

Mensajes:

Evento	Trama
Pulsador presionado	EVT:e_stop
Pulsador liberado (o rearme)	EVT:e_stop_rel
Heartbeat periódico (cada 1 s)	STS:OK

Evento	Trama
Lectura de ADC (cada 500 ms)	DAT:adc=NNNN

La Pill espera ACK:estop tras cada EVT:e_stop. Si no lo recibe en 500 ms, reenvía.

Reacción del robot (cátedra): se detiene inmediatamente al recibir e_stop. No reanuda hasta recibir e_stop_rel.

Evidencia esperada: LED cambia de verde fijo a rojo destellante al presionar; buzzer suena; monitor serie muestra EVT:e_stop; robot se detiene. Al liberar, LED vuelve a verde, buzzer calla, robot reanuda.

11.2 Variante 2: Deadman switch / hombre muerto

Concepto: el operador debe mantener presionado un pulsador para que el robot funcione. Al soltarlo, la Pill ordena parada inmediata. Mientras el pulsador está presionado, un potenciómetro define la velocidad del robot.

Periféricos:

- **EXTI (botón):** pulsador que debe mantenerse presionado. Flanco descendente (soltar) dispara el evento deadman. Anti-rebote por timer.
- **LED:** verde con brillo PWM proporcional a la velocidad comandada mientras el botón está presionado; rojo fijo al soltar.
- **Buzzer:** frecuencia proporcional a la velocidad (tono más agudo = más rápido) mientras el botón está presionado; pitido de alarma al soltar.
- **ADC:** lee un potenciómetro en PA0 que define la velocidad deseada (0–100 %).
- **PWM:** controla brillo del LED (proporcional a velocidad) y frecuencia del buzzer (tono variable).

Mensajes:

Evento	Trama
Botón presionado (operador presente)	EVT:deadman_ok
Velocidad (cada 250 ms, mientras presionado)	DAT:spd=NN
Heartbeat (cada 1 s, mientras presionado)	STS:ALIVE
Botón liberado (operador ausente)	EVT:deadman

La Pill espera ACK:ok tras EVT:deadman. Reenvía cada 500 ms hasta recibirlo.

Reacción del robot (cátedra): opera a la velocidad reportada mientras recibe ALIVE. Se detiene inmediatamente al recibir deadman.

Evidencia esperada: LED se apaga al soltar el botón; buzzer calla y emite pitido de alarma; monitor muestra transición deadman_ok → deadman; robot frena al soltar.

11.3 Variante 3: Caja de comandos física

Concepto: la Blue Pill implementa una caja de comandos con un pulsador que cicla entre cuatro modos de operación del robot: PATROL → STOP → DEPLOY → RETURN. Un potenciómetro ajusta un parámetro auxiliar que se envía junto con el comando.

Periféricos:

- **EXTI (botón):** cada presión válida (con anti-rebote) cicla al modo siguiente.
- **LED:** patrón PWM distinto por modo: PATROL = destello lento (1 Hz), STOP = fijo, DEPLOY = destello rápido (4 Hz), RETURN = doble destello.
- **Buzzer:** código de pitidos por modo: 1 pitido = PATROL, 2 = STOP, 3 = DEPLOY, 4 = RETURN.

- **ADC:** lee un potenciómetro en PA0. Su valor se mapea a 0–255 y se envía como parámetro del comando.
- **PWM:** controla el patrón de destello del LED por modo.

Mensajes:

Evento	Trama
Cambio a modo PATROL	EVT:cmd_patrol + STS:mode=PATROL + DAT:param=NNN
Cambio a modo STOP	EVT:cmd_stop + STS:mode=STOP + DAT:param=NNN
Cambio a modo DEPLOY	EVT:cmd_deploy + STS:mode=DEPLOY + DAT:param=NNN
Cambio a modo RETURN	EVT:cmd_return + STS:mode=STOP + DAT:param=NNN

La Pill espera ACK:ok tras cada EVT.

Reacción del robot (cátedra): PATROL = recorrido; STOP = detenido; DEPLOY = posición de guardia o inspección; RETURN = vuelve a base.

Evidencia esperada: LED cambia patrón al presionar el botón; buzzer emite código distinto por modo; monitor muestra EVT:cmd_XXX; robot cambia visiblemente de comportamiento en cada modo.

11.4 Variante 4: Señalización industrial

Concepto: la Blue Pill actúa como baliza o torreta de señalización industrial (luz + sonido) que refleja el estado del robot. Recibe el estado desde el bridge y lo traduce a patrones luminosos y acústicos normalizados. Un sensor de luz ambiente ajusta automáticamente la intensidad de los LED.

Periféricos:

- **LED (2 o RGB):** patrones de torreta: verde fijo = RUN, amarillo fijo = STOP, rojo destello lento (1 Hz) = ERROR, rojo destello rápido (4 Hz) = ESTOP.
- **Buzzer:** silencio en RUN; tono intermitente en STOP y ERROR; tono continuo en ESTOP.
- **ADC:** lee un sensor de luz ambiente (fotorresistor o LDR, provisto por la cátedra) conectado a PA0. El valor se usa para ajustar automáticamente la intensidad PWM de los LED: a más luz ambiente, más brillo.
- **PWM (2 canales):** controla brillo de LED verde/amarillo y rojo con corrección por luz ambiente.

Mensajes:

Evento	Trama
Recibe RUN del bridge	CMD:mode=RUN
Recibe STOP del bridge	CMD:mode=STOP
Recibe ERROR del bridge	CMD:mode=ERROR
Recibe ESTOP del bridge	CMD:mode=ESTOP
Heartbeat periódico (cada 1 s)	STS:OK
Lectura de luz ambiente (cada 1 s)	DAT:adc=NNNN
Timeout sin CMD (> 3 s)	ERR:timeout

Esta variante es la única que **recibe CMD del bridge**. La Pill no controla al robot; solo refleja su estado.

Reacción del robot (cátedra): el bridge envía CMD mode=XXX según el estado real del robot. La Pill actúa como display remoto.

Evidencia esperada: LED cambia color y patrón al cambiar el estado del robot; brillo se ajusta al tapar o destapar el sensor de luz; buzzer cambia según el modo; monitor muestra CMD:mode=XXX.

11.5 Variante 5: Zona segura con proximidad

Concepto: la Blue Pill monitorea un sensor de proximidad analógico y define tres zonas concéntricas alrededor del robot: segura, advertencia y peligro. Al ingresar a cada zona, notifica al robot y activa la señalización correspondiente.

Periféricos:

- **ADC:** lee un sensor de proximidad o distancia analógico (provisto por la cátedra; puede ser IR, ultrasónico con salida analógica, etc.). El mapeo entre valor ADC y zona es configurable por software.
- **LED (2 o RGB):** verde fijo en zona segura; amarillo con destello PWM (50 %, 2 Hz) en advertencia; rojo con destello PWM (50 %, 4 Hz) en peligro.
- **Buzzer:** silencio en zona segura; tono intermitente lento en advertencia; tono continuo en peligro.
- **PWM (2 canales):** controla destello de LED amarillo y rojo según la zona activa.

Mensajes:

Evento	Trama
Ingresa en zona segura	EVT:zone_safe
Ingresa en zona de advertencia	EVT:zone_warn
Ingresa en zona de peligro	EVT:zone_danger
Lectura de ADC (cada 250 ms)	DAT:adc=NNNN

La Pill espera ACK:ok tras cada EVT:zone_danger. Reenvía cada 500 ms hasta recibirlo.

Reacción del robot (cátedra): operación normal en zona segura; reduce velocidad en advertencia; se detiene por completo en peligro.

Evidencia esperada: LED cambia de verde → amarillo → rojo al acercar un objeto al sensor; buzzer pasa de silencio → intermitente → continuo; monitor muestra EVT:zone_XXX; robot frena al entrar en zona de peligro.

11.6 Tabla resumen de variantes

#	Nombre	Gatillo principal	Tipos de mensaje	Periféricos extra
1	Parada de emergencia	Pulsador (EXTI)	EVT, STS, DAT, ACK	—
2	Deadman switch	Pulsador sostenido (EXTI)	DAT, EVT, STS, ACK	—
3	Caja de comandos	Pulsador cíclico (EXTI)	EVT, STS, DAT, ACK	—
4	Señalización industrial	CMD desde bridge	CMD, STS, DAT, ERR	Sensor de luz
5	Zona segura	Sensor proximidad (ADC)	EVT, DAT, ACK	Sensor de distancia

12 Entregables

Cada grupo debe entregar:

1. **Repositorio o carpeta del proyecto** con el código fuente completo del firmware, incluyendo Makefile o CMakeLists.txt funcional.
2. **Informe técnico en PDF** con la siguiente estructura:
 - Objetivo del TP y variante elegida.
 - Esquema de conexión (qué pin de la Blue Pill va a cada periférico).

- Tabla de configuración de periféricos (registros/API, parámetros, cálculos).
 - Desarrollo por etapas con evidencia (capturas, fotos, diagramas).
 - Respuestas a todas las preguntas obligatorias de cada etapa.
 - Descripción del escenario de falla probado en la Etapa 4.
 - Breve reflexión técnica del grupo sobre lo aprendido.
3. **Video de demostración** (máximo 3 minutos) mostrando:
- El montaje físico completo.
 - La operación de los periféricos (LED, buzzer) ante estímulos.
 - Las tramas en el monitor serie.
 - Si hay acceso al robot: la reacción del robot ante los mensajes de la Blue Pill.

Extensión sugerida del informe: 4 a 8 páginas de desarrollo, más anexos si aplica.

13 Evaluación

Puntaje total: **10 puntos**. La evaluación se distribuye en cuatro grandes categorías:

- **Funcionamiento técnico:** protocolo, comunicación, periféricos y correcta implementación de la variante.
- **Informe:** claridad, trazabilidad, respuestas a las preguntas obligatorias y evidencia documentada.
- **Exposición / demo:** presentación del montaje, operación en vivo y capacidad de explicar decisiones de diseño.
- **Seguridad y evidencia:** cumplimiento de las reglas de seguridad, manejo de fallas y evidencia concreta de integración.

El bridge ROS 2 y la reacción del robot son provistos por la cátedra. Los estudiantes no son evaluados por implementar el bridge, sino por generar tramas correctas y producir evidencia concreta de integración.

14 Checklist de entrega

Antes de entregar, verificá que:

- ☐ El parser recibe y valida correctamente al menos los seis tipos de mensaje (CMD, DAT, EVT, STS, ACK, ERR).
- ☐ El parser rechaza tramas con checksum incorrecto, longitud incorrecta o caracteres no permitidos.
- ☐ El parser se resincroniza correctamente tras ruido o trama incompleta.
- ☐ El ADC lee valores coherentes al variar el sensor físico (0–4095).
- ☐ El LED cambia su comportamiento (brillo, patrón, color) según lo descrito en tu variante.
- ☐ El buzzer emite sonidos diferenciados según lo descrito en tu variante.
- ☐ Las tramas generadas son sintácticamente válidas (verificadas con monitor serie).
- ☐ Los payloads numéricos mantienen ancho fijo con relleno de ceros.
- ☐ La tabla de configuración de periféricos está completa con cálculos de PSC, ARR y frecuencias.
- ☐ El informe responde todas las preguntas obligatorias de las cuatro etapas.
- ☐ El video muestra el sistema completo funcionando de principio a fin.
- ☐ Si tu variante recibe CMD, la Pill responde correctamente a los cambios de estado.
- ☐ Si tu variante espera ACK, implementaste reenvío ante timeout.